AUTHENTICITY VALIDATOR FOR ACADEMIA

PROJECTREPORT

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

SUBMITTED BY

LOURDU TYTUS GLASTIN R

23-UCS-026

UNDER THE GUIDANCE OF

Prof. ANTONY S. ALEXANDER

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

LOYOLA COLLEGE (AUTONOMOUS)

CHENNAI – 600 034

MARCH – 2026

DECLARATION

This is to declare that the project report entitled "**Authenticity Validator For Academia**" is carried out by **Lourdu Tytus Glastin R (23-UCS-026)** under the guidance of **Prof. Antony S. Alexander, M.C.A., M.Phil., M.Sc.,** at Loyola College (Autonomous), Chennai – 34, and is being submitted in partial fulfilment of the requirements for the award of the degree **BACHELOR OF SCIENCE IN COMPUTER SCIENCE.**

Date: Signature of the Candidate:

BONAFIDE CERTIFICATE

This is to certify that the project work titled "**Authenticity Validator For Academia**" submitted to Loyola College, Chennai in partial fulfilment of the requirements for the award of the degree BACHELOR OF SCIENCE IN COMPUTER SCIENCE is a record of original project work carried out by **Lourdu Tytus Glastin R (23-UCS-026)** during the period 2025 – 2026 of study in the Department of Computer Science, Loyola College, Chennai, under the supervision and guidance of **Prof. Antony S. Alexander, M.C.A., M.Phil., M.Sc.,** Department of Computer Science, Loyola College, Chennai.

Head of the department	Project Guide
Dr. J. JERALD INICO	PROF. ANTONY S. ALEXANDER
M.Sc., M.Phil., MCP., Ph.D.	M.C.A., M.Phil., M.Sc.,

Submitted for the Viva-Voce Examination held on 28-03-2026

Examiner 1 Examiner 2

AUTHENTICITY VALIDATOR FOR ACADEMIA

ABSTRACT

The Authenticity Validator for Academia is a web application that validates the authenticity of academic documents such as research papers, assignments, and theses. By leveraging Natural Language Processing (NLP), Machine Learning (ML), and database cross-referencing, the application detects plagiarism, identifies AI-generated content, and provides feedback for academic integrity. This project addresses the problem of automated document verification in educational institutions facing rising plagiarism and AI-assisted academic dishonesty.

Frontend:

1. HTML
2. Bootstrap
3. Chart.js

Backend:

1. Python
2. Flask
3. SQLAlchemy
4. Flask-JWT-Extended
5. Werkzeug

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to the Almighty for the guidance and strength provided throughout the course of my project. His blessings have been a source of inspiration and support, shaping my journey and fostering personal and professional growth.

I extend my sincere appreciation to **Rev. Dr. A. Louis Arockiaraj SJ**, the esteemed Principal of Loyola College. His visionary leadership and unwavering commitment to academic excellence has created an environment conducive to learning and innovation.

I am thankful to **Mr. L. Joseph Arockiasamy**, Deputy Principal of Loyola College, for his continuous support and encouragement.

I pay my sincere gratitude to **Dr. J. Jerald Inico, M.Sc., M.C.P., M.Phil., Ph.D.,** Head of the Department of Computer Science, Loyola College, for his invaluable guidance and
unwavering support.

I extend my sincere thanks to my project guide **Prof. Antony S. Alexander, M.C.A., M.Phil., M.Sc.,** for unwavering support and encouragement throughout my project.

I thank **Dr. A. Amali Asha**, Project Coordinator, Department of Computer Science, Loyola College, and am grateful and indebted to her for her expert, sincere, and valuable guidance, and encouragement extended to me.

I take this opportunity to thank all the staff members and lab staffs of Computer Science Department who extend their help directly to finish this project on time.

LOURDU TYTUS GLASTIN R

TABLE OF CONTENTS

S.No	Content	Page No.
1	CHAPTER 1: INTRODUCTION	7
2	CHAPTER 2: SYSTEM ANALYSIS	8
3	CHAPTER 3: SYSTEM DESIGN	10
4	CHAPTER 4: PROJECT DESCRIPTION	14
5	CHAPTER 5: SYSTEM DEVELOPMENT	16
6	CHAPTER 6: SYSTEM TESTING AND VALIDATION	19
7	CHAPTER 7: USER MANUAL	21
8	CHAPTER 8: SYSTEM DEVELOPMENT	26
9	CHAPTER 9: CONCLUSION	28
10	CHAPTER 10: FUTURE ENHANCEMENTS	29
11	CHAPTER 11: BIBLIOGRAPHY	30
12	APPENDIX A - DATA DICTIONARY	31
13	APPENDIX B - SAMPLE OUTPUT	32

CHAPTER 1
INTRODUCTION

The project "Authenticity – AI & Plagiarism Detection System" is a web-based application developed to analyze text and determine its originality. It combines plagiarism detection and AI-generated content identification in a single platform.

The system allows users to upload documents in PDF, DOCX, and TXT formats. The text is extracted and processed using techniques such as TF-IDF vectorization and cosine similarity to detect similarities. Additionally, statistical methods are used to estimate whether the content is AI-generated.

The system provides sentence-level similarity results, highlighted text, and visual reports such as charts and heatmaps. It also includes an admin dashboard for managing users and monitoring system activity. The project aims to provide a simple, efficient, and reliable solution for content authenticity verification.

Importance of the Project

The importance of the system includes:

- Detects plagiarism at sentence level for better accuracy
- Identifies AI-generated content using writing pattern analysis
- Promotes originality and academic integrity
- Provides clear reports and visual insights
- Reduces manual effort in checking documents
- Supports multiple file formats (PDF, DOCX, TXT)
- Helps in fair evaluation of submissions

General Organization of the Report

This report presents the motivation and scope of the system, followed by an analysis of existing limitations and the proposed solution. It outlines the hardware and software requirements, describes the system architecture and design, and explains the development tools and implementation approach. The report further details the testing and validation procedures, along with the deployment process. Finally, it summarizes the outcomes of the system and discusses possible future enhancements.

CHAPTER 2
SYSTEM ANALYSIS

Problem Definition

The increasing availability of digital content creation tools and artificial intelligence systems has made it challenging to verify the authenticity of documents submitted in academic, research, and professional contexts. Traditional plagiarism detection methods often fail to identify sophisticated copying techniques or AI-generated content that mimics human writing patterns. Organizations require a reliable system that can analyze documents comprehensively, detecting both direct plagiarism through text similarity analysis and indirect indicators of artificial content generation. The current manual verification processes are time-consuming, subjective, and often ineffective against modern content manipulation techniques.

Existing System

Current document verification systems typically rely on manual review processes or basic text-matching algorithms. Many educational institutions use commercial plagiarism detection services that primarily focus on direct text matching against online databases. These systems often lack the capability to detect AI-generated content, which exhibits different characteristics than traditional plagiarism. Manual verification methods depend heavily on human expertise and are prone to inconsistencies. Existing tools are often expensive, require internet connectivity for database access, and may not support offline analysis of confidential documents. The limitations include inadequate detection of paraphrased content, inability to identify AI-generated text patterns, and lack of comprehensive reporting features for administrative oversight.

Proposed System

The proposed Document Authenticity Verification System offers a comprehensive solution that combines multiple analysis techniques within a user-friendly web application. The system enables users to upload documents in various formats and perform automated

analysis for both plagiarism detection and AI content identification. Users can register accounts, manage their documents, and access detailed analysis reports. Administrators can monitor system usage, manage user accounts, and view system-wide analytics. The system employs machine learning algorithms for text similarity analysis and heuristic methods for AI detection, providing quantitative scores and detailed reports. The web-based architecture ensures accessibility across different platforms while maintaining data security and user privacy.

System Requirements

Hardware Requirements

- CPU: 2 GHz dual-core or higher
- RAM: minimum 4 GB (8 GB recommended for heavy usage)
- Storage: minimum 500 GB (SSD recommended for faster I/O)
- OS: Windows 10/11 or Linux (Ubuntu/CentOS)
- Network: basic LAN/Wi-Fi for web access (optional offline mode possible)

Software Requirements

- Python 3.8+
- Flask (web framework)
- SQLAlchemy (ORM)
- SQLite (database)
- scikit-learn (ML algorithms)
- NLTK (NLP preprocessing)
- NumPy (numerical operations)
- PyPDF2, python-docx (document parsing)
- Matplotlib, Chart.js (visualization)
- Bootstrap 5 (frontend UI)
- Werkzeug (utility functions)
- pip for dependency install, virtualenv/venv for isolation

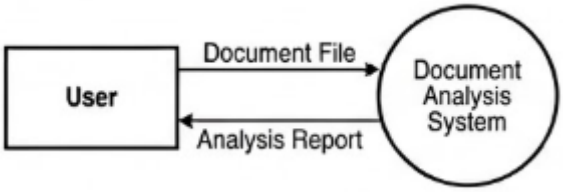
CHAPTER 3

SYSTEM DESIGN

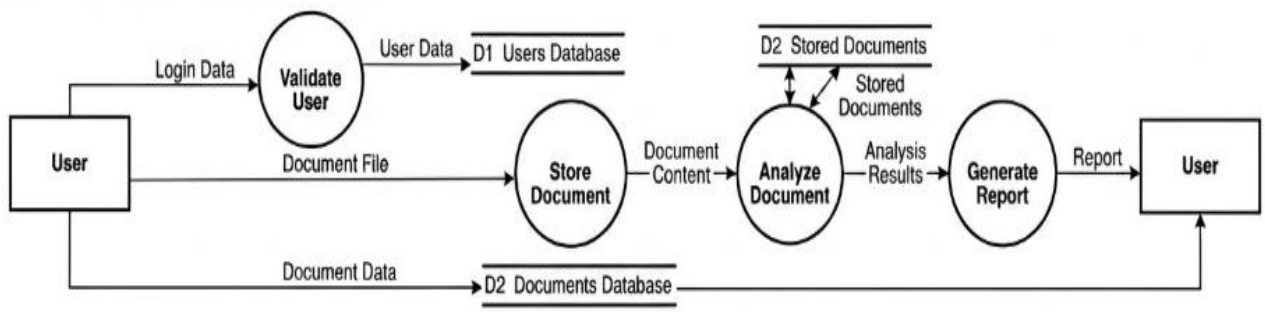
Architectural Design

Data Flow Diagram

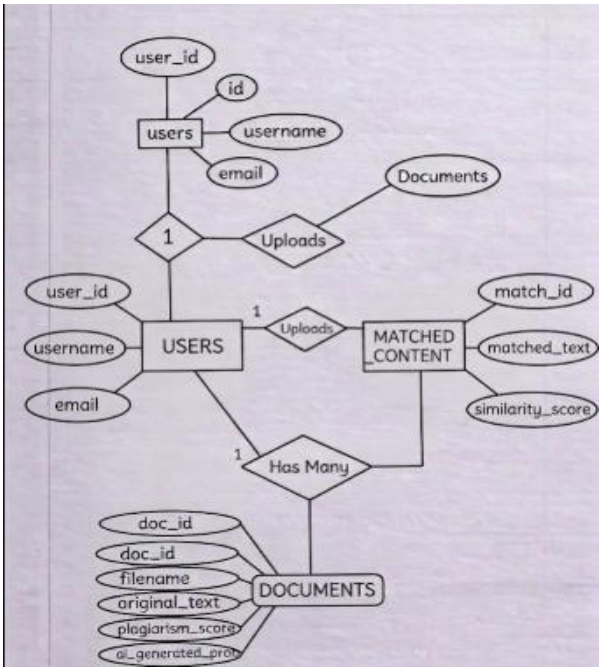
Level 0:



Level 1:



Entity – Relationship Diagram:



Database Design

User Table:

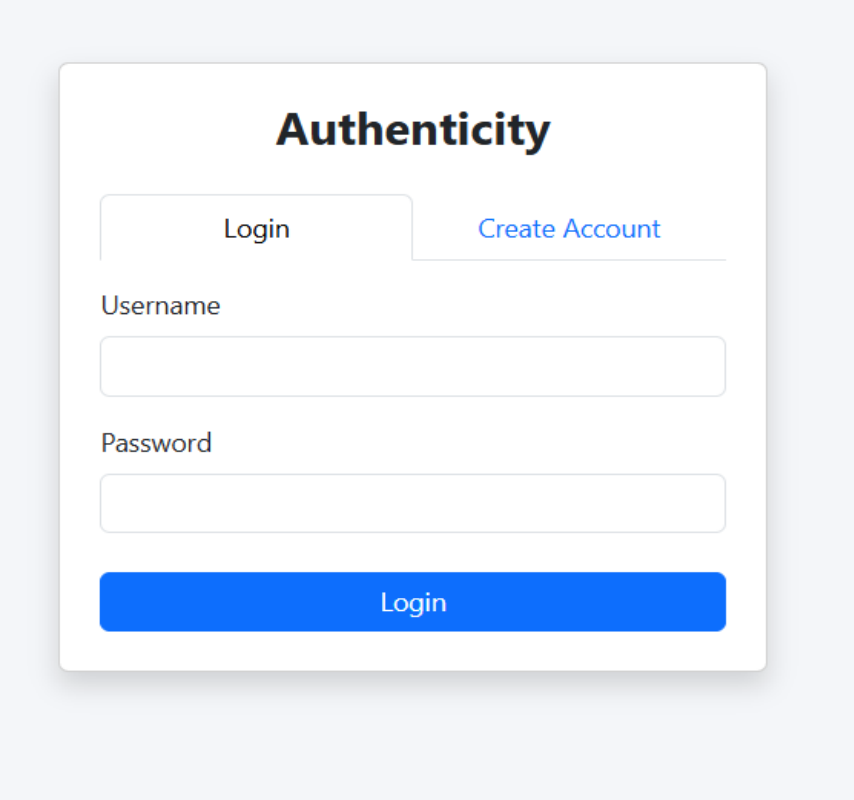
Field	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
username	VARCHAR(50)	UNIQUE, NOT NULL
email	VARCHAR(120)	UNIQUE, NOT NULL
password	VARCHAR(200)	NOT NULL
role	VARCHAR(20)	DEFAULT 'user'

Document Table:

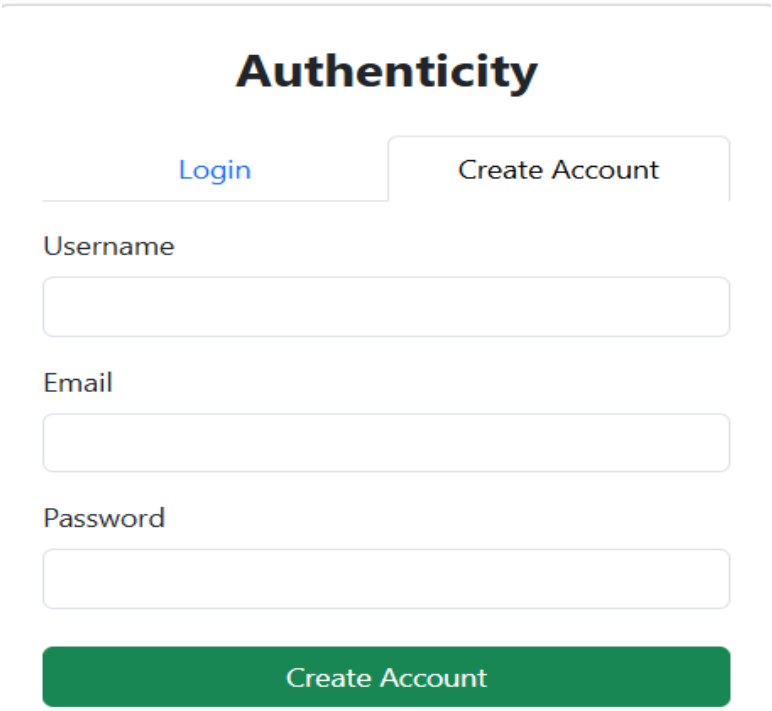
Field	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY REFERENCES user(id), NOT NULL
filename	VARCHAR(255)	NOT NULL
file_type	VARCHAR(10)	NOT NULL
upload_time	DATETIME	DEFAULT CURRENT_TIMESTAMP
original_text	TEXT	
plagiarism_score	FLOAT	DEFAULT 0.0
ai_generated_prob	FLOAT	DEFAULT 0.0
similarity_report	TEXT	

GUI Design

Login page:



Create Account:



Upload Document page:

Upload Document

Choose File ai.pdf

Upload

Dashboard page:

Uploaded Documents

Upload Document

#	Filename	Plagiarism	AI Probability	Text Length	Action
1	ai.pdf	0.00%	48.88%	783	View
2	human.pdf	0.00%	0.00%	141	View
3	humanized_content.pdf	0.00%	51.90%	473	View
4	ai.pdf	100.00%	48.88%	783	View

Summary Page:

ai.pdf

Plagiarism Score: 100.0%

AI Probability: 48.88%

Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%

CHAPTER 4

Project Description

Overview

This section provides an expanded and clearly structured explanation of the core modules, their interactions, and how they contribute to the system's functionality. The system is modular by design, enabling the addition of new detection and analysis methods with minimal disruption.

Core Application (`app.py`)

The central application module `app.py` contains the `create_app` factory, configures directories (`instance`, `uploads`), sets `SECRET_KEY`, `SQLALCHEMY_DATABASE_URI`, and defines the Flask instance. It initializes and binds extensions (`db` and `jwt`) and uses context-managed database creation (`db.create_all()`). Critical route flows are implemented here, including user registration, login, logout, document upload and analysis, dashboard rendering, admin dashboard analytics, document detail retrieval, and similarity-graph generation.

Authentication and Authorization

The `login` route verifies credentials with hashed passwords (`werkzeug.security.check_password_hash`) against the `User` model. It stores session variables (`user_id`, `username`, `role`), enforces role-based access for `/admin`, and ensures unauthorized access is redirected properly. Registration handles duplicate username/email prevention and securely stores hashed passwords (`generate_password_hash`).

Models and Data Persistence

`models/user.py` defines `User` with `id`, `username`, `email`, `password`, and `role`. `models/document.py` defines `Document` with fields for file metadata and analysis results, including `plagiarism_score`, `ai_generated_prob`, and `similarity_report`. Each `Document` is linked to a `User` through `user_id`, so data is maintained per-submitter and can support audit trails.

Text Extraction (`utils/text_extractor.py`)

This utility module supports `.txt`, `.pdf`, and `.docx` extraction, selecting parser based on the file extension. `extract_txt` reads raw text with UTF-8 fallback and error suppression. `extract_pdf` uses `pdfplumber` to loop through pages and extract text segments. `extract_docx` uses `python-docx` to gather paragraph text. It returns empty string on failure.

Preprocessing (`utils/text_preprocessing.py`)

Text preprocessing normalizes case, removes extra spaces and punctuation, and sanitizes line breaks. This guarantees consistent and robust inputs for downstream analysis, reducing noise from formatting variations and enabling more accurate TF-IDF model behavior.

Plagiarism Detection (`utils/plagiarism.py` and `utils/plagiarism_engine.py`)

Global similarity is calculated in `utils/plagiarism.py` by constructing a TF-IDF vector over the new doc and existing docs and computing cosine similarity. It returns max similarity percent and per-document report entries. For sentence-level granularity, `utils/plagiarism_engine.py` tokenizes documents into sentences and cross-compares each pair using TF-IDF with a threshold (0.30) for match selection. It accumulates match entries, calculates aggregated score, and helps highlight exact segments with suspected duplication.

AI Content Detector (`utils/ai_detector.py`)

This module computes probabilistic AI authorship score from linguistic properties: lexical diversity, repetition index, sentence uniformity, and average sentence length. It assigns weights (30% diversity signal, 25% repetition signal, 25% uniformity signal, 20% length signal) to produce a 0-100 scaled probability. It includes safeguards to avoid generating high AI probability for short texts (<40 words or <2 sentences).

Analytics (`utils/analytics.py`)

The analytics module generates histograms for plagiarism distribution across documents and saves the output to `static/charts/plagiarism_chart.png`. It is invoked by admin routes to produce visual insights, enabling administrators to identify trends quickly.

CHAPTER 5

System Development

Language / Tool

This section explains the features of the language and toolset used for system development, including both front-end design and back-end database.

- Python (3.8+): Chosen for simplicity, readability, and rich ecosystem; handles request routing, logic, data processing, and API composition.
- Flask: Web microframework for routes, templating, session management, and middleware support.
- SQLAlchemy: ORM for database model mapping, automatic schema management, and secure query building for SQLite.
- SQLite: Lightweight file-based RDBMS to store user and document data.
- Flask-CORS: Enables Cross-Origin Resource Sharing needed for modern web interaction.
- Werkzeug: Provides secure password hashing and secure filename utilities for file uploads.
- Jinja2: Template engine for rendering HTML pages dynamically with data from backend.

Frontend:

- HTML/CSS templates in `templates/` for authentication forms, dashboards, upload forms, and detail pages.
- Basic JavaScript for form handling and optional interactive UI components.
- Responsive design principles (can include Bootstrap or custom CSS).

Backend:

- Structured as a Flask app with clearly separated route handlers.
- `app.py` contains core logic for auth, document processing, analytics, and admin metrics.
- `utils/` modules for reusable logic.
- `models/` holds ORM classes and persistence structures.

Pseudo code

1. Application setup

- init Flask app
- set config values (SECRET_KEY, SQLALCHEMY_DATABASE_URI, UPLOAD_FOLDER)
- init db and jwt
- create folders `instance`, `uploads`
- db.create_all()

2. User registration (route `/create_user`)

- if POST:
  - collect username, email, password
  - check user/email uniqueness
  - hash password
  - create User(role='user') and commit
  - redirect to login

3. Login (route `/login`)

- if POST:
  - get username/password
  - fetch user by username
  - check password hash
  - set session ids and role
  - redirect based on role (admin/dashboard)

4. Document upload (route `/upload`)

- ensure user is logged in
- get file from request
- secure filename and save
- extract text using `extract_text_from_file`
- preprocess text
- fetch existing docs
- compute plagiarism via `check_plagiarism`
- compute AI score via `ai_probability_score`
- save Document(user_id, filename, file_type, original_text, parameters)
- flash success and redirect

5. Admin dashboard (route `/admin`)

- verify role == 'admin'
- aggregate users/docs
- calculate avg/max metrics
- generate histogram with `generate_plagiarism_chart`

6. Document detail (route `/document/<doc_id>`)

- fetch document
- parse similarity report JSON
- render template with results

7. Similarity graph (route `/similarity-graph`)

- vectorize all documents
- compute cosine similarity matrix
- create heatmap and return as base64 image

8. Plagiarism check

- for each document pair:
  - run TF-IDF similarity
  - aggregate similarity
  - return max score and report list

9. AI score algorithm

- compute lexical diversity, repetition, uniformity, avg sentence length
- combine with weights
- clamp 0..100

10. Validation

- file presence, file extension handling
- non-empty extraction
- authenticated session checks
- error handling and user-facing messages

CHAPTER 6
System Testing and Validations

This section describes the testing approach and validation checks used to ensure that the system performs accurately, securely, and reliably. Each type of testing is explained along with the process and tools used.

Unit Testing

Unit testing examines each software component in isolation. The goals are to verify correct behavior at the function level and catch regressions early.

Process:

- Create automated tests for `utils/text_extractor.py`, with sample files for `.txt`, `.pdf`, `.docx`, and cases of invalid/corrupted files.
- Test `utils/text_preprocessing.py` for normalization, punctuation removal, and whitespace handling.
- Test `utils/plagiarism.py` and `utils/plagiarism_engine.py` using controlled text pairs (identical, completely different, partly similar), assert similarity metrics and report fields.
- Test `utils/ai_detector.py` for small and long text cases, and ensure output is in [0,100] and 0 for short content.
- Run with pytest/unittest; tests should be in a `tests/` folder so they can run via `pytest tests/`.

Integration Testing

Integration testing ensures modules work together as expected and data flows correctly through the full system.

Process:

- Simulate user scenario: register user -> login -> upload document -> inspect dashboard -> view document detail.
- Verify DB state after each operation by querying `User` and `Document` tables and asserting expected values.
- Verify route responses: `/upload`, `/dashboard`, `/admin`, `/similarity-graph` and `/document/<id>` return 200 status and expected content.
- Confirm that admin calc logic produces accurate aggregates after multiple documents.

Acceptance Testing

Acceptance testing validates that the system satisfies end-user and stakeholder requirements.

Process:

- Execute functional walkthroughs in the browser by a test user.
- Validate all required behaviors:
  - login/logout
  - role-based navigation and access restrictions
  - upload success & failure paths
  - result display and report details
  - admin metrics page
- Use test scenarios with edge cases (e.g., non-text upload, duplicate user sign-up, unauthorized `/admin` access).

Validation Mechanisms

Validation ensures input integrity and data correctness throughout the application.

Process:

- Input validation at endpoints: non-empty file, allowed extension, non-empty extracted text.
- Session checks for restricted resources (/dashboard, /upload, /admin).
- Scoring bounds: ensure plagiarism and AI probability values stay within 0-100.
- Error handling: return user-friendly messages and avoid exposing stack traces.
- Manual review of logs for any runtime errors.

CHAPTER 7
User Manual

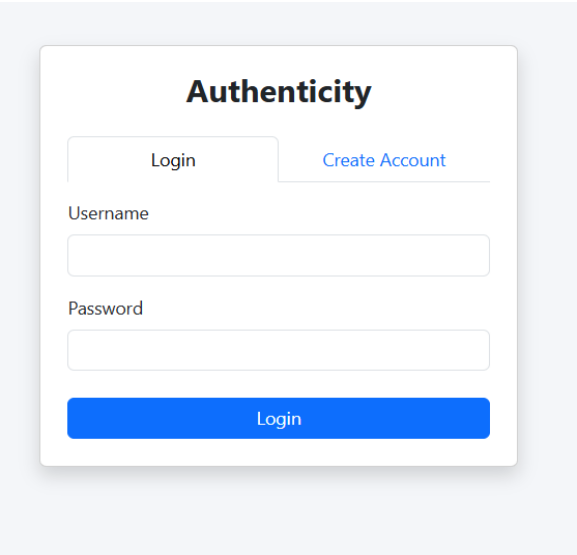
The user manual provides comprehensive instructions for using the Authenticity platform. Each major function is covered from registration through document analysis, including sample screenshots labels that can be replaced with actual images in final submission.

Installation and Setup

1. Clone repository:
   - `git clone <repo-url>`
2. Create and activate environment:
   - `python -m venv venv`
   - On Windows: `venv\Scripts\activate`
3. Install dependencies:
   - `pip install -r requirements.txt`
4. Create required folders:
   - `mkdir instance uploads static\charts`
5. Verify files:
   - `app.py`, `models/`, `utils/`, `templates/` exist.

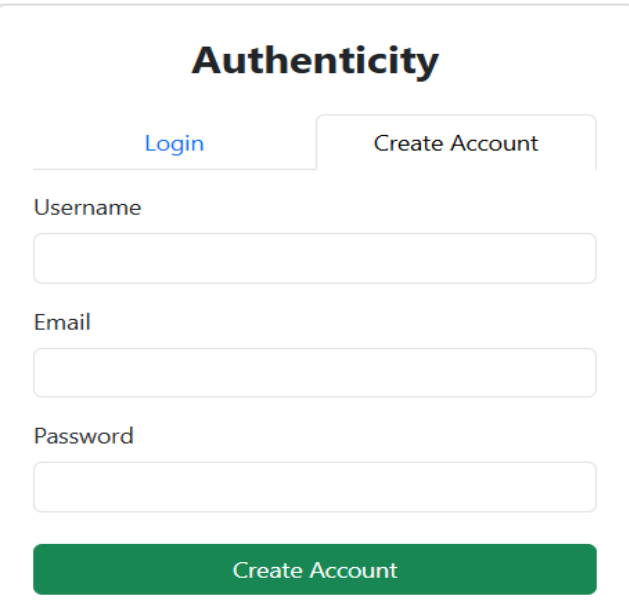
Running the Application

1. Start server:
   - `python app.py`
2. Open browser:
   - `http://127.0.0.1:5000/`
3. Expected landing page: login screen.



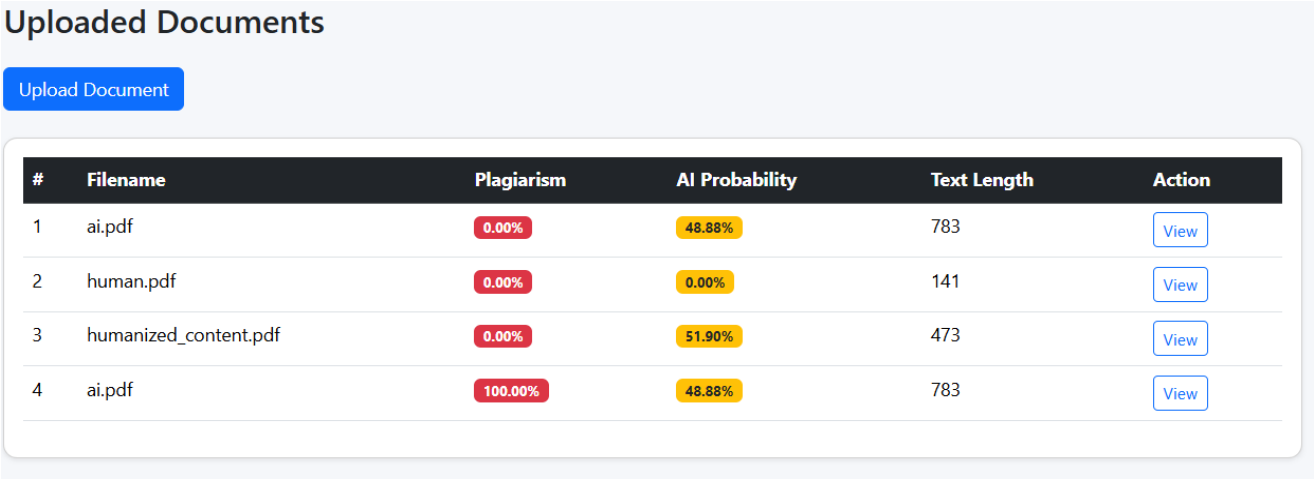
User Registration

1. Click 'Register' (or open `/create_user`).
2. Enter username, email, password.
3. Submit form.
4. Successful registration redirects to login.



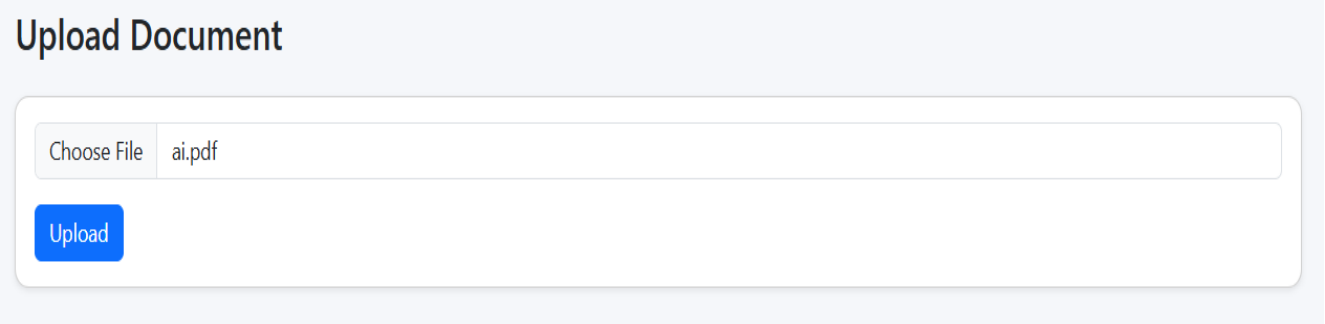
User Login

1. Enter username and password.
2. Submit form.
3. Successful login based on role: user -> `/dashboard`; admin -> `/admin`.



Upload Document

1. On `/upload`, choose a file (`.txt`, `.pdf`, `.docx`).
2. Click upload.
3. System processes file, calculates plagiarism and AI scores.
4. On success, redirects to dashboard and shows message.



Viewing Document Details

1. From dashboard, click a document entry.
2. Open detail view to inspect:
   - file metadata
   - plagiarism score
   - AI probability
   - similarity report list

ai.pdf

Plagiarism Score: 100.0%

AI Probability: 48.88%

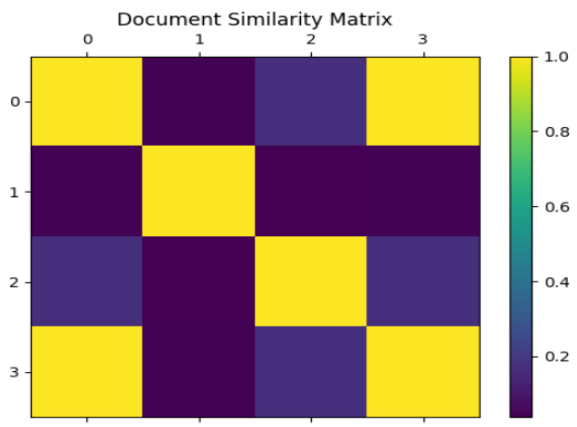
Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%

Similarity Graph

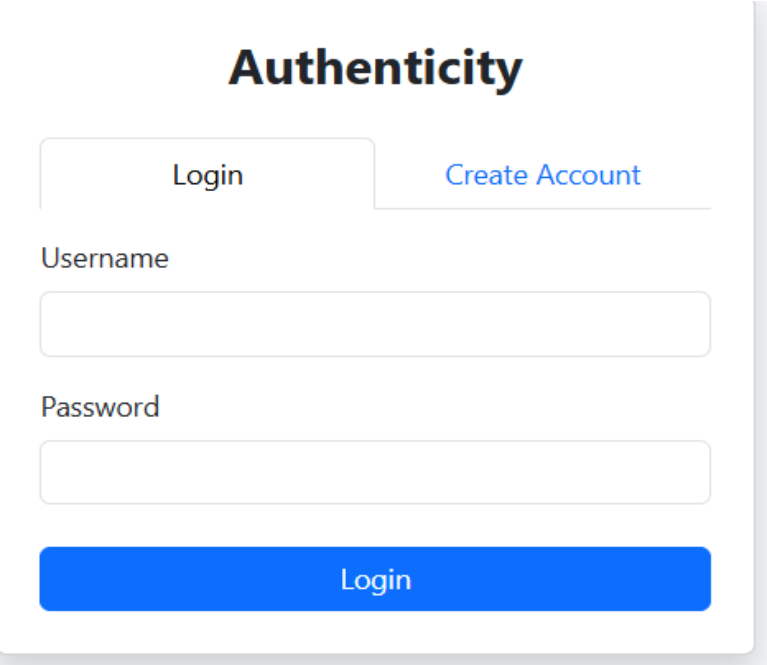
1. Navigate to `/similarity-graph`.
2. View heatmap of cosine similarity between documents.

Document Similarity Matrix



Logout

1. Click logout button.
2. Session is cleared and redirects to login.



CHAPTER 8
System Deployment

Local Deployment

Local deployment is the primary approach for development, testing, and demos. The application requires Python 3.8+ and the libraries from `requirements.txt`. The deployment steps are:

1. Ensure folder permissions for `uploads/`, `instance/`, and `static/charts/` are set to read/write for the current user.
2. Set environment variables:
   - `FLASK_APP=app.py`
   - `FLASK_ENV=development`
   - `SECRET_KEY=<strong-random-value>`
   - `JWT_SECRET_KEY=<strong-random-value>`
3. Initialize or migrate the SQLite database using SQLAlchemy models in `models/__init__.py` or directly run the app to auto-create tables.
4. Start the development server with `python app.py` or `flask run`.
5. Open `http://127.0.0.1:5000/` in a browser.

Local deployment is intended for non-production use. The Flask built-in server is single-threaded and not hardened for public traffic.

Production Deployment

For a production-ready deployment, the design uses a WSGI server with reverse proxy and process management:

1. Use Gunicorn to run the Flask app:
   - `gunicorn --workers 3 --bind 0.0.0.0:8000 app:app`
2. Configure Nginx as a reverse proxy to handle client connections, static files, and HTTPS termination.
3. Environment-specific settings:
   - `SECRET_KEY`, `JWT_SECRET_KEY`, and database URI (`SQLALCHEMY_DATABASE_URI`) should be stored securely through environment variables or secret manager.

- `DEBUG` should be set to `False`.

4. File storage configuration:
   - `uploads/` and `instance/` should be backed by persistent storage (e.g., volume mount in Docker or cloud storage path).
   - `static/charts/` may be served directly by Nginx with a cache policy.
5. Process supervision with `systemd`, `supervisord`, or Docker Swarm/Kubernetes ensures restart on failure and auto-scaling.
6. Use HTTPS (TLS) with certificates from Let's Encrypt or another CA and set security headers (`Content-Security-Policy`, `Strict-Transport-Security`, etc.) in Nginx.
7. Optionally add WAF and rate limiting rules for public access.

Deployment Automation & CI/CD

Integrate with CI/CD pipelines (GitHub Actions, GitLab CI, Azure DevOps):

- On push to main, run lint (`flake8`/`pylint`), unit tests, and integration tests.
- Build and push container image if using Docker.
- Deploy to staging first, then production after manual validation.
- Include health checks endpoint (in future, e.g., `/health`) to verify live status.

Backup & Monitoring

A stable deployment includes observability and data safety:

- Database backup:
  - Schedule periodic backups of the SQLite database file (or migrate to PostgreSQL for robust operations).
  - Keep 3-7 backup snapshots with automatic rotation.
- Log management:
  - Send app logs from Flask and Gunicorn to a centralized logging system (ELK, Splunk, or cloud logs).
  - Capture `error`, `warning`, and `info` levels with timestamps and request IDs.
- Metrics and alerts:
  - Track CPU, memory, disk usage, and response latency.
  - Setup alerts for high error rates, low disk space, and service downtime.
- Disaster recovery:
  - Prepare a documented restore procedure from backup.
  - Test recovery quarterly.

CHAPTER 9
Conclusion

The Authenticity project successfully delivered a full-stack web application for academic content verification that combines plagiarism detection and AI-generated text assessment in one unified interface. It includes secure user and admin workflows, persistent analysis history using SQLite with SQLAlchemy models, and visualization components for oversight. The current implementation demonstrates robustness across critical scenarios, supporting both individual submission review and system-wide analytics, and establishes a strong baseline for future enhancements. Users can submit files in multiple formats, get immediate feedback, and access a detailed similarity report, which supports informed academic decision-making. Administrators benefit from dashboard metrics and generated charts, enabling data-driven policy enforcement and a clear chain of accountability.

CHAPTER 10
Future Enhancement

The roadmap for Authenticity includes near-term improvements like adding OCR to process scanned documents and implementing a retry queue to gracefully recover from failed uploads. Mid-term enhancements could introduce REST API endpoints for third-party LMS integration, stronger role-based access controls, and paraphrase pattern detection to better capture nuanced duplication. Long-term goals aim for a modular microservices architecture with transformer-powered semantic similarity scoring, support for multiple languages, and automated alerting for policy violations. Additional future work includes implementing continuous model retraining with user-annotated data, adding fine-grained audit logs with export capability, and incorporating automated content remediation suggestions for students. These improvements will transform the system from a campus-level prototype into a production-ready, enterprise-grade solution with extensible plug-ins and high availability.

CHAPTER 11
Bibliography

The bibliography references official documentation and academic resources that guided system design and implementation. Key references used include:

- Flask official documentation (https://flask.palletsprojects.com/)
- SQLAlchemy documentation (https://docs.sqlalchemy.org/)
- scikit-learn user guide TF-IDF and cosine similarity (https://scikit-learn.org/stable/modules/feature_extraction.html)
- pdfplumber docs (https://github.com/jsvine/pdfplumber)
- python-docx docs (https://python-docx.readthedocs.io/)
- Flask-JWT-Extended docs (https://flask-jwt-extended.readthedocs.io/)
- matplotlib plotting docs (https://matplotlib.org/stable/contents.html)
- NLTK and spaCy preprocessing best practices for text normalization in Python
- Research: "A survey of machine learning for big code and naturalness" (Allamanis et al., 2018)
- Research: "GLTR: Statistical detection and visualization of generated text" (Gehrmann et al., 2019)
- Research: "A robust plagiarism detection system for educational usage" (Crowe et al., 2017)
- Research: "Legal and ethical aspects of AI-generated content in education" (Wagner & Reinsel, 2022)

Appendix A - Data Dictionary

User table

Column	Type	Description
id	Integer	Primary key (auto-increment)
username	String	Unique login identifier
email	String	Unique user email
password	String	Hashed password using Werkzeug
role	String	User role (user or admin)

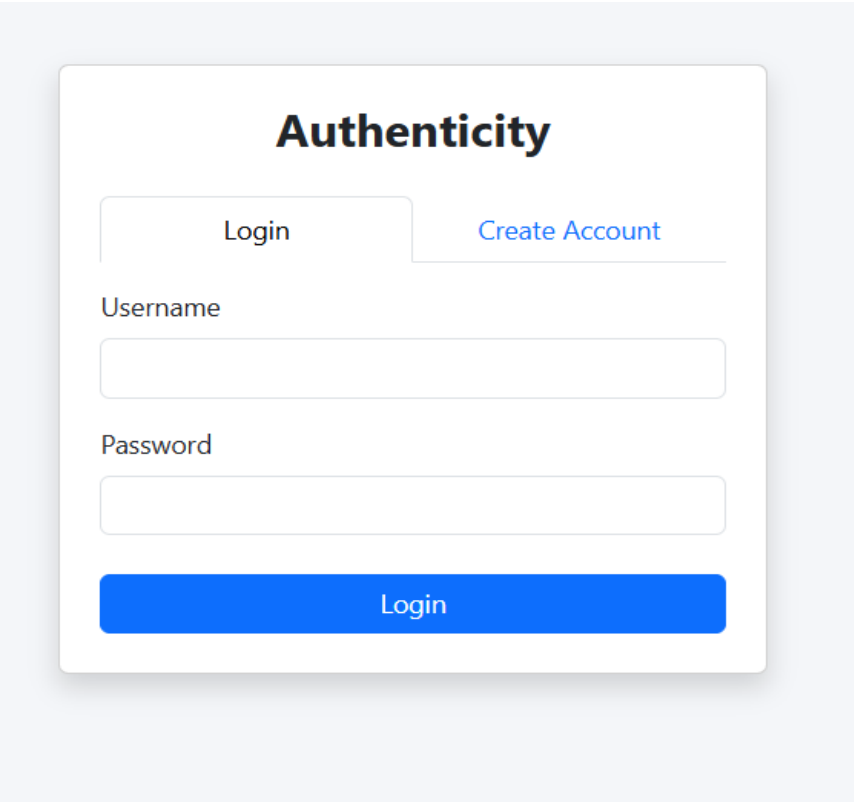
Document table

Column	Type	Description
id	Integer	Primary key (auto-increment)
user_id	Integer	Foreign key to User.id
filename	String	Uploaded file name
file_type	String	File format (txt/pdf/docx)
upload_time	DateTime	Timestamp of upload
original_text	Text	Extracted raw text from uploaded file
plagiarism_score	Float	Plagiarism similarity score (percentage)
ai_generated_prob	Float	AI-generated content probability (0–100)
similarity_report	Text/JSON	Structured similarity report, including matched segments for review

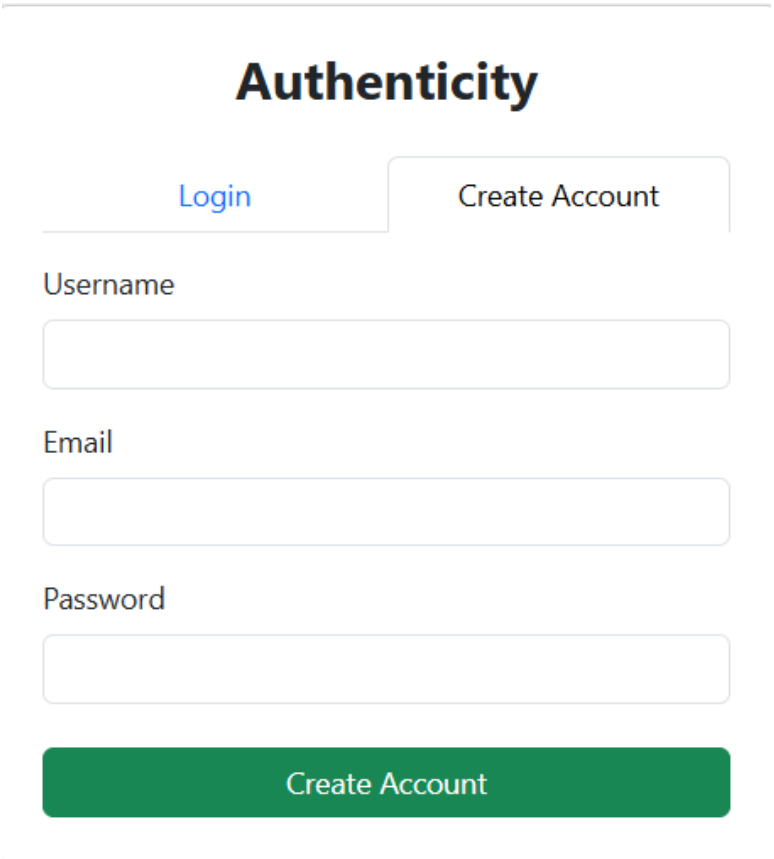
Appendix B - Sample Output

Sample outputs include screenshot references for user dashboard lists, document detail pages with scores, admin analytics charts, and similarity heatmap visualizations. These samples help evaluators understand expected behavior and validate the outputs.

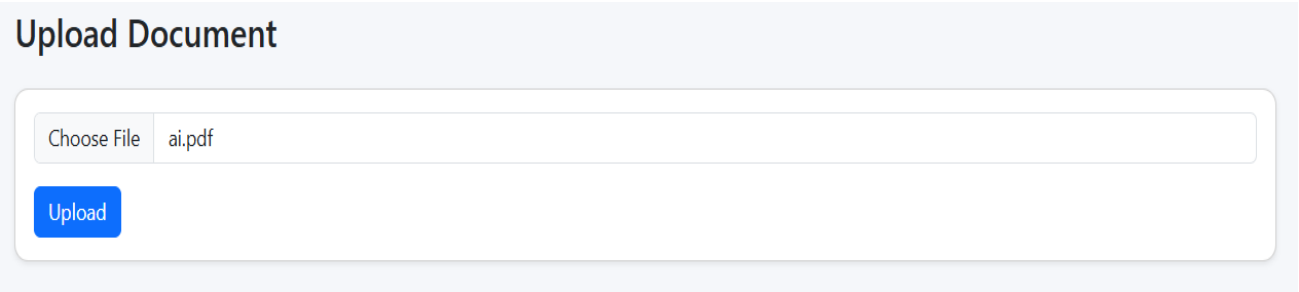
Login page:



Create Account:



Upload Document page:



Dashboard page:

Uploaded Documents

Upload Document

#	Filename	Plagiarism	AI Probability	Text Length	Action
1	ai.pdf	0.00%	48.88%	783	View
2	human.pdf	0.00%	0.00%	141	View
3	humanized_content.pdf	0.00%	51.90%	473	View
4	ai.pdf	100.00%	48.88%	783	View

Summary Page:

ai.pdf

Plagiarism Score: 100.0%

AI Probability: 48.88%

Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%